Object Oriented Programming for Data Processing
17 February 2025

Your exam material (code files, plots, datafiles, etc.) must be submitted via google classroom by 15:15 as a single `zip` file.

C++ evaluation will be based on: correct syntax, proper return types and arguments of functions, data members and class interfaces, setters/getters, correct mathematical expressions, comments throughout the code, separation of class implementations and interfaces.

Python evaluation will be based on: correct syntax, avoiding C-style loops, using Python features in general, comments throughout the notebook/scripts, labels, legends and plot styling and clarity in general.

Part 1 – C++

Consider the following abstract class:

```
class Function {
  public:
    Function(const std::string& name) { name_ = name; };
    // Value of the function at a point in 3D space
    virtual double value(double x, double y, double z) const = 0;
    // Derivative of the function with respect to var at a point in 3D space
    virtual double deriv(double x, double y, double z, char var) const = 0;
    virtual std::string name() const { return name_; };
  private:
    std::string name_;
};
```

Implement a concrete class `Polynomial`, that inherits from `Function`, to represent and handle polynomials of three real independent variables (x, y, and z) of at most second degree ($a_0 + a_1x + a_2y + a_3z + a_4x^2 + a_5y^2 + a_6z^2 + a_7xy + a_8xz + a_9yz$). The class must provide a regular constructor that takes as input a `std::vector` with 10 `doubles` that represent the coefficients $\{a_i\}$, and a setter that also takes a 10-dimensional `std::vector` as an argument.

Implement a class `VectorField` to represent real-valued vector fields in 3D. It must provide the following.

- A regular constructor that takes 3 `Function` pointers, to be stored as private members.
- A method to evaluate the vector field at a specific x, y, and z.
- Methods to determine the divergence ($\nabla\cdot$) and curl ($\nabla\times$) of the a specific x, y, and z.

Finally, write a short application to show how `VectorField` works.

Part 2 – Python

Build a class to handle the simulation of random walks, where the direction of each *step* is drawn from an isotropic distribution and the length of each *step* is drawn from the Pareto II distribution, which has probability density function

$$p(r, a) = \begin{cases} \dfrac{a}{r^{a+1}} & r \geq 1 \\ 0 & r < 1 \end{cases},$$

where $a > 0$ is known as shape parameter. This distribution is directly accessible with `numpy.random.pareto(a, size=None)`.

For the purpose of this implementation, all random walks start in the origin and they can take place in 2 or 3 dimensions.

Use a Python notebook or Python scripts to complete the following tasks.

- Provide the ability to initialize instances of the class by passing the shape parameter, the number of walkers `Nexp`, the number of steps `Nsteps` each walker takes (this is the same number for all walkers), and the number of dimensions simualted (2 or 3).
- Provide a method to generate the `Nexp` random walks.
- Provide a method to save the simulated walks in a single `pickle` file.
- Provide the ability to display the results of a simulation.

Once your class is implemented, show a 2D and a 3D example with 10 random walkers (the choices of other parameters are up to you).